

Panduan Kode (untuk Mahasiswa)

Cross-Layer Grace-Convergence Controller:
Menyambung Terminasi Pod Kubernetes dengan Konvergensi Cluster Elixir/BEAM

Dokumentasi teknis proyek `01_cross_layer_grace_controller`

Dihasilkan otomatis — versi panduan kode

Abstract

Dokumen ini menjelaskan **kode** dari sebuah *controller fault-tolerance lintas-lapis*: sistem yang membuat Kubernetes dan runtime Elixir/BEAM “berbicara” satu sama lain saat sebuah pod dimatikan. Tujuannya menutup apa yang kami sebut *grace gap*: celah antara waktu tetap yang diberikan Kubernetes sebelum mematikan paksa sebuah pod, dan waktu yang sebenarnya dibutuhkan runtime untuk memindahkan (handoff) state in-memory-nya ke node lain. Panduan ini ditulis dengan **bahasa yang mudah** untuk mahasiswa S1 tahun ke-3 Teknik Informatika: setiap istilah teknis dijelaskan saat pertama kali muncul, disertai analogi sehari-hari, cuplikan kode asli, dan **langkah persis** untuk menjalankan seluruh eksperimen (RQ1–RQ8). Semua nama fungsi, file, dan angka diambil langsung dari kode sumber di `01_cross_layer_grace_controller/`.

Contents

1	Pendahuluan: Masalah “Grace Gap”	2
1.1	Gambaran singkat	2
1.2	Apa itu terminasi pod Kubernetes?	2
1.3	Apa itu handoff BEAM?	3
1.4	Mengapa grace period <i>tetap</i> itu salah	3
1.5	Apa yang dilakukan controller ini	3
2	Struktur Folder Proyek	4
3	Library yang Dipakai dan Perannya	6
4	Penjelasan Modul-per-Modul	7
4.1	GraceConvergence — modul puncak	7
4.2	GraceConvergence.Application — pohon supervisi	7
4.3	GraceConvergence.Grace — kebijakan grace	9
4.4	GraceConvergence.Probe — sensor konvergensi	9
4.5	GraceConvergence.StatefulWorker — satu proses stateful	10
4.6	GraceConvergence.Workers — buat, hitung, cari worker	11
4.7	GraceConvergence.Handoff — pemindah state	11
4.8	GraceConvergence.Shutdown — hook terminasi adaptif	12
4.9	GraceConvergence.ProbeHTTP — endpoint HTTP	13
4.10	GraceConvergence.Operator — operator Kubernetes	13
4.11	GraceConvergence.Presence — workload realistis (RQ8)	14

4.12	GraceConvergence.Harness — driver eksperimen	14
4.13	Berkas pengujian (test/)	14
5	Cara Kerja Langkah-demi-Langkah	15
5.1	Alur A — Eksperimen drain 2-node lokal (tanpa Kubernetes)	16
5.2	Alur B — Loop Kubernetes nyata	16
6	Bagaimana Kode Menjawab RQ1–RQ8	17
7	Panduan Menjalankan Eksperimen untuk Semua RQ	18
7.1	RQ1 & RQ2 — correctness & efficiency (tabel dua beban)	19
7.2	RQ2, RQ3, RQ4, RQ5 — sweep, rollout, overhead, sensitivitas	19
7.3	Statistik (confidence interval) — pengulangan	19
7.4	RQ6 — scalability ($ H $ 1k..40k)	19
7.5	RQ8 — realistic workload (Phoenix.Presence)	19
7.6	RQ7 & Kubernetes — latensi nyata + fail-safe	20
8	Tabel Pemetaan Ringkas (RQ → Skrip → CSV → Figur)	20
9	Glosarium Singkat	21

1 Pendahuluan: Masalah “Grace Gap”

1.1 Gambaran singkat

Bayangkan sebuah aplikasi yang menyimpan banyak data penting *di memori* (RAM), bukan di database: misalnya ribuan “ruang chat”, sesi game, atau keranjang belanja yang sedang aktif. Aplikasi seperti ini dijalankan di atas **Kubernetes** (sistem yang mengatur banyak container/aplikasi di banyak mesin) dan ditulis dengan **Elixir** yang berjalan di atas mesin virtual bernama **BEAM** (mesin virtual milik bahasa Erlang/Elixir, terkenal untuk sistem yang harus selalu hidup).

Masalah muncul ketika Kubernetes perlu mematikan salah satu pod (unit aplikasi terkecil di Kubernetes), misalnya saat **rolling update** (memperbarui aplikasi versi-demi-versi tanpa mematikan semuanya sekaligus). Kalau state di RAM belum sempat dipindahkan ke node lain sebelum pod dimatikan paksa, **state itu hilang**. Itulah masalah inti yang ditangani proyek ini.

1.2 Apa itu terminasi pod Kubernetes?

Ketika Kubernetes ingin menghentikan sebuah pod, ia tidak langsung “mencabut steker”. Urutannya:

1. Kubernetes mengirim sinyal **SIGTERM** ke proses di dalam pod. **SIGTERM** adalah sinyal sistem operasi yang artinya “tolong berhenti dengan rapi”. Proses *boleh* menanganinya dan melakukan pembersihan.
2. Kubernetes menunggu sejumlah detik tetap, yang disebut **termination grace period** (di manifest Kubernetes namanya `terminationGracePeriodSeconds`). Anggap ini “waktu toleransi” yang diberikan untuk berberes.
3. Setelah waktu itu habis, Kubernetes mengirim **SIGKILL**. **SIGKILL tidak bisa** ditangkap atau diabaikan — proses langsung mati seketika. Apa pun yang belum selesai, hilang.

Analogi. Bayangkan Anda penyewa kamar kos dan diminta pindah. Pemilik kos berkata: “Saya beri waktu *tepat 30 menit* untuk mengangkut semua barang; lewat dari itu, semua yang tersisa saya buang.” Kalau barang Anda sedikit, 30 menit lebih dari cukup. Tetapi kalau Anda baru saja belanja banyak (beban tinggi), 30 menit bisa tidak cukup, dan barang sisa terbuang. “Waktu toleransi” (grace period) yang *tetap* itulah sumber masalahnya.

1.3 Apa itu handoff BEAM?

Di Elixir/BEAM, satuan kerja terkecil adalah **proses** (sangat ringan, bukan proses sistem operasi; satu node BEAM bisa menjalankan jutaan). Setiap proses stateful menyimpan datanya sendiri di memori. Ketika sebuah node akan pergi, datanya harus dipindahkan ke node yang bertahan. Pemindahan state dari node yang akan mati ke node lain inilah yang disebut **handoff**.

Di proyek ini handoff berarti: *baca state proses* $\rightarrow$ *matikan proses lokal* $\rightarrow$ *hidupkan kembali proses itu di node “survivor” (yang bertahan) dengan state yang sama*. Kalau handoff belum selesai saat SIGKILL datang, proses-proses yang belum sempat dipindah (beserta datanya) lenyap.

1.4 Mengapa grace period *tetap* itu salah

Inti masalahnya: berapa detik yang “cukup” itu **bergantung pada beban saat itu**, bukan angka konstan.

- **Terlalu pendek** (misal tetap 30 detik): saat beban tinggi, handoff terpotong SIGKILL, dan state hilang. Ini pelanggaran *correctness* (kebenaran).
- **Terlalu panjang** (misal tetap 300 detik untuk berjaga-jaga): setiap pod butuh waktu lama untuk benar-benar mati, sehingga setiap rolling update jadi sangat lambat. Ini pemborosan (*efficiency*).

Celah antara grace tetap dan kebutuhan handoff sebenarnya inilah yang disebut **grace gap**. Informasi untuk menghitung deadline yang tepat (berapa proses tersisa, secepat apa bisa dipindah) ada di *runtime* (BEAM), tetapi kontrol yang menegakkan deadline ada di *orkestrator* (Kubernetes), dan **tidak ada yang menghubungkan keduanya**. Proyek ini membangun jembatan itu.

1.5 Apa yang dilakukan controller ini

Controller menghitung grace period secara *adaptif* dari sinyal runtime. Kebijakannya:

$$g = \text{clamp}\left(T_d + \frac{|H|}{\rho} + T_c + \sigma, g_{\min}, g_{\max}\right)$$

dengan arti tiap simbol:

- $|H|$ = **backlog handoff**: berapa proses stateful yang masih harus dipindahkan.
- ρ (dibaca “rho”) = **laju handoff**: berapa proses per detik yang bisa dipindahkan.
- T_c = **waktu konvergensi**: waktu agar cluster “tenang” kembali (membership stabil, registry sinkron) setelah node pergi.
- T_d = **waktu drain**: waktu menyelesaikan permintaan yang sedang berjalan sebelum state dipindahkan.
- σ (dibaca “sigma”) = **margin keamanan**: bumper tambahan agar tidak pas di tepi.
- $g_{\min}, g_{\max}$ = batas bawah dan atas, supaya grace tidak terlalu kecil maupun terlalu besar.

Suku kunci $|H|/\rho$ secara intuitif adalah “jumlah pekerjaan / kecepatan = waktu”, yaitu perkiraan lama handoff. **Invariant keselamatan** (aturan yang harus selalu benar agar tidak ada state hilang):

$$g \geq T_d + \frac{|H|}{\rho} + T_c.$$

Selama grace yang diberikan tidak lebih kecil dari kebutuhan sebenarnya, handoff tidak akan terpotong.

Analogi. Kembali ke kasus pindah kos: controller ini seperti seorang asisten cerdas yang melihat berapa banyak barang Anda sekarang ($|H|$), seberapa cepat tukang angkut bekerja (ρ), lalu berkata kepada pemilik kos: “Beri dia tepat $|H|/\rho$ menit, ditambah sedikit bumper.” Tidak terlalu mepet (barang aman) dan tidak terlalu lama (kamar cepat kosong).

2 Struktur Folder Proyek

Proyek tinggal di `01_cross_layer_grace_controller/`. Dokumen yang sedang Anda baca berada di folder baru `code_guide/`, sejajar dengan `code/` dan `paper/`. Berikut pohon direktori inti (folder hasil build seperti `_build/` dan `deps/` dihilangkan):

```
01_cross_layer_grace_controller/
|-- README.md          # status proyek + cara build (baca paling awal)
|-- CONTEXT.md         # catatan konvensi (jangan diregres)
|-- OUTLINE.md         # kerangka paper
|-- code/              # ARTIFACT + harness eksperimen
|   |-- README.md
|   |-- DESIGN.md      # analisis -> requirement -> desain -> validasi
|   |-- Dockerfile     # 1 image; GRACE_ROLE memilih app vs operator
|   |-- app/           # proyek Elixir/OTP 'grace_convergence'
|       |-- README.md
|       |-- mix.exs     # daftar dependency + metadata proyek
|       |-- mix.lock    # versi dependency yang dikunci
|       |-- config/     # config.exs, dev.exs, test.exs, prod.exs
|       |-- lib/
|           |-- grace_convergence.ex # modul puncak (delegasi)
|           |-- 'grace_convergence/ # semua modul inti
|               |-- application.ex  # pohon supervisi (boot)
|               |-- grace.ex        # kebijakan grace = clamp(...)
|               |-- probe.ex        # sensor konvergensi (EWMA)
|               |-- stateful_worker.ex # 1 proses stateful
|               |-- workers.ex      # buat/hitung/cari worker
|               |-- handoff.ex      # pindahkan state ke survivor
|               |-- shutdown.ex     # hook terminasi adaptif
|               |-- probe_http.ex   # /probe /healthz /drain
|               |-- operator.ex     # operator Kubernetes (kubect1)
|               |-- presence.ex     # Phoenix.Tracker (RQ8)
|               |-- harness.ex      # driver eksperimen
|       |-- harness/              # skrip .exs untuk menjalankan eksperimen
|           |-- run.exs           # tabel 2-beban (RQ1/RQ2)
|           |-- sweep.exs        # sweep + rollout + overhead + sensitivitas
|           |-- repeats.exs      # pengulangan N=10 / N=5 (confidence interval)
|           |-- scale.exs        # |H| 1k..40k (RQ6)
|           |-- 'presence.ex     # konvergensi Phoenix.Presence (RQ8)
|       |-- scripts/
|           |-- 'cluster_probe.exs # menonton handoff 2-node secara manual
|           |-- 'test/
|               |-- grace_test.exs # 6 unit test untuk Grace
|               |-- 'cluster_handoff_test.exs # 2 test integrasi cluster
|-- k8s/              # manifest + skrip Kubernetes
|   |-- rbac.yaml     # izin (ServiceAccount + Role)
|   |-- app.yaml      # Deployment app + Service + PDB + preStop
```

```

| | |-- operator.yaml      # Deployment operator
| | |-- netem.sh          # injeksi latensi nyata tc netem (RQ7)
| | '-- faults.sh         # injeksi fault (fail-safe)
| '-- analysis/
|     '-- plot.py          # data/*.csv -> figures/*.pdf (matplotlib)
|-- data/                  # hasil eksperimen (CSV; hanya data nyata)
|-- figures/               # diagram (.puml/.dot) + plot (.pdf)
|-- paper/                 # manuskrip LaTeX (main.tex, references.bib, ...)
|-- code_guide/            # << dokumen INI (panduan kode)
|-- cover_letter/          # surat pengantar submisi
'-- declarations/         # deklarasi (etika, dsb.)

```

Tabel berikut merangkum **tanggung jawab** tiap folder dan file penting.

Path	Tanggung jawab
README.md (top)	Ringkasan status proyek, keputusan desain, cara build. Baca paling awal.
CONTEXT.md	Catatan konvensi penulisan & gotcha eksperimen yang tidak boleh diregres.
code/DESIGN.md	Alur analisis → requirement (FR/NFR) → desain → rencana validasi (V1–V7).
code/app/	Proyek Elixir/OTP <i>grace_convergence</i> : seluruh logika inti.
code/app/mix.exs	Definisi proyek + daftar dependency (Horde, libcluster, Bandit, Plug, Jason, phoenix_pubsub).
code/app/config/	Konfigurasi per-lingkungan: <i>config.exs</i> (umum), <i>test.exs</i> , <i>prod.exs</i> (strategi libcluster Kubernetes).
code/app/lib/grace_convergence/	11 modul inti (grace, probe, worker, handoff, shutdown, operator, dll.). Dirinci di Bagian 4.
code/app/harness/	Skrip <i>.exs</i> yang <i>menjalankan</i> eksperimen dan menulis CSV ke <i>data/</i> .
code/app/scripts/cluster_probe.exs	Alat manual untuk menonton satu handoff 2-node secara langsung.
code/app/test/	<i>grace_test.exs</i> (6 unit test kebijakan grace) dan <i>cluster_handoff_test.exs</i> (2 test integrasi cluster nyata).
code/k8s/	Manifest Kubernetes (<i>rbac.yaml</i> , <i>app.yaml</i> , <i>operator.yaml</i>) + skrip <i>netem.sh</i> & <i>faults.sh</i> .
code/analysis/plot.py	Mengubah <i>data/*.csv</i> menjadi <i>figures/*.pdf</i> memakai matplotlib.
code/Dockerfile	Satu image multi-stage; variabel <i>GRACE_ROLE</i> memilih peran <i>app</i> atau <i>operator</i> .
data/	Berkas CSV hasil eksperimen — <i>hanya data nyata</i> , tidak ada angka fiktif.
figures/	Sumber diagram (<i>.puml/.dot</i>) dan plot hasil (<i>.pdf</i>).
paper/	Manuskrip LaTeX (memakai kelas Wiley; bukan dokumen ini).
code_guide/	Dokumen panduan kode ini (kelas <i>article</i> standar).

3 Library yang Dipakai dan Perannya

Sebelum masuk ke tabel, beberapa istilah dasar yang sering muncul:

GenServer

“Server process” standar di OTP (kerangka kerja Erlang/Elixir untuk sistem tahan-gagal). Sebuah GenServer memiliki state privat dan menangani pesan *satu per satu*. Caller bicara via `GenServer.call` (sinkron: menunggu balasan) atau `GenServer.cast` (asinkron: kirim lalu lupa). Banyak modul di proyek ini adalah GenServer.

Supervisor

Proses “pengawas” yang menjalankan proses-proses lain (anak) dan me-*restart* mereka bila mati. Filosofi Erlang: “let it crash” — biarkan proses mati, lalu supervisor menghidupkannya lagi ke kondisi bersih. **DynamicSupervisor** adalah supervisor yang anaknya dibuat sesuai kebutuhan saat runtime (bukan daftar tetap saat boot).

CRDT

Conflict-free Replicated Data Type — struktur data yang bisa diubah secara lokal di banyak node, lalu otomatis “menyatu” (merge) ke hasil yang sama di semua node tanpa perlu koordinasi pusat. Cocok untuk sistem terdistribusi. Horde memakai CRDT.

Operator

Pola di Kubernetes: sebuah program yang terus “mengawasi” kondisi cluster dan “mendamaikan” (reconcile) kondisi nyata ke kondisi yang diinginkan. Operator di proyek ini membaca probe tiap pod lalu menambal `terminationGracePeriodSeconds`.

Library / Tool	Peran dalam proyek
Elixir / OTP	Bahasa dan platform dasar. Menyediakan GenServer (Probe, Shutdown, StatefulWorker, Operator, Presence), Supervisor & DynamicSupervisor (WorkerSup menjadi tuan rumah worker), serta Application (boot pohon supervisi).
Horde (~ 0.9)	Registry terdistribusi berbasis delta-CRDT + dukungan handoff. Horde.Registry memberi tiap worker <i>identitas unik se-cluster</i> sehingga node mana pun bisa mencari worker berdasarkan kunci, walau worker pindah node.
libcluster (~ 3.3)	Membentuk cluster BEAM secara otomatis. Lokal memakai strategi Gossip; di Kubernetes memakai Cluster.Strategy.Kubernetes (menemukan pod via headless service). Lihat <code>config/prod.exs</code> .
Bandit (~ 1.5)	Web server untuk menyajikan endpoint HTTP probe (<code>/probe</code> , <code>/healthz</code> , <code>/drain</code>). Dijalankan dari pohon supervisi di <code>application.ex</code> .
Plug (~ 1.16)	Lapisan “router” HTTP. GraceConvergence.ProbeHTTP memakai Plug.Router untuk memetakan path ke handler.
Jason (~ 1.4)	Encode/decode JSON. Probe membalas dalam JSON; operator mem-parsing JSON pod & probe, dan menyusun patch JSON untuk kubectl.
phoenix_pubsub (~ 2.1)	Menyediakan Phoenix.PubSub dan Phoenix.Tracker — mesin CRDT di balik Phoenix.Presence. Dipakai sebagai <i>workload realistis</i> untuk mengukur waktu konvergensi nyata T_c (RQ8).
kubectl System.cmd	via Operator memanggil biner kubectl dari Elixir (<code>System.cmd/2</code>) untuk mendaftarkan pod dan menambal Deployment. Jadi operator ini adalah <i>GenServer biasa yang memanggil kubectl</i> — bukan framework Bonny.

Library / Tool	Peran dalam proyek
kind	“Kubernetes IN Docker” — menjalankan cluster Kubernetes di dalam container Docker di satu mesin. Dipakai untuk eksperimen V3 dan RQ7.
kubectl	CLI Kubernetes untuk apply manifest, melihat log, scale, dsb.
docker	Membangun image aplikasi dan menjalankan node kind.
Graphviz (dot)	Mesin tata-letak grafik; dipakai PlantUML untuk merender diagram dan oleh berkas .dot.
matplotlib	Library Python untuk membuat plot; <code>analysis/plot.py</code> memakainya untuk membuat seluruh figur <code>eval_*.pdf</code> .
tc / netem	<code>tc</code> (traffic control) + modul <code>netem</code> milik kernel Linux untuk menyuntikkan latensi jaringan buatan. Dipakai <code>k8s/netem.sh</code> untuk RQ7.

4 Penjelasan Modul-per-Modul

Semua modul inti ada di `code/app/lib/grace_convergence/`. Untuk tiap modul: tugas satu kalimat, fungsi-fungsi kunci, posisinya dalam keseluruhan, dan cuplikan kode *asli* yang pendek.

Sebelum masuk ke detail tiap modul, Gambar 1 memberi gambaran besar *bagaimana modul-modul ini saling memanggil* ketika satu drain (handoff) berlangsung. Bacalah dari atas ke bawah mengikuti waktu: Shutdown meminta angka ke Probe, menghitung grace lewat Grace, lalu menyuruh Handoff memindahkan worker melalui Workers.

4.1 GraceConvergence — modul puncak

Tugas: pintu masuk ringkas; hanya mendelegasikan tiga fungsi populer ke modul yang sebenarnya.

Posisi: memudahkan dipakai dari REPL (`iex`).

```
defdelegate reading(), to: GraceConvergence.Probe
defdelegate drain_and_await(), to: GraceConvergence.Shutdown
defdelegate start_many(n), to: GraceConvergence.Workers
```

`defdelegate` berarti “panggilan ke fungsi ini sebenarnya diteruskan ke modul lain”. Jadi `GraceConvergence.reading()` sama dengan `GraceConvergence.Probe.reading()`.

4.2 GraceConvergence.Application — pohon supervisi

Tugas: menentukan proses apa saja yang dijalankan saat aplikasi boot, sesuai *peran* (app atau operator).

Posisi: “akar” dari segalanya; tanpa modul ini tidak ada yang hidup.

Fungsi kunci: `start/2` (callback OTP), `role/0` (membaca `GRACE_ROLE`).

```
children =
  case role() do
    :operator ->
      [GraceConvergence.Operator]          # pod operator: hanya koordinator

    :app ->
      cluster_children(cfg) ++             # libcluster (pembentuk cluster)
      [
        {Horde.Registry, [name: GraceConvergence.Registry, keys: :unique, members: :auto]},
```

Interaksi antar-modul: satu drain (handoff)

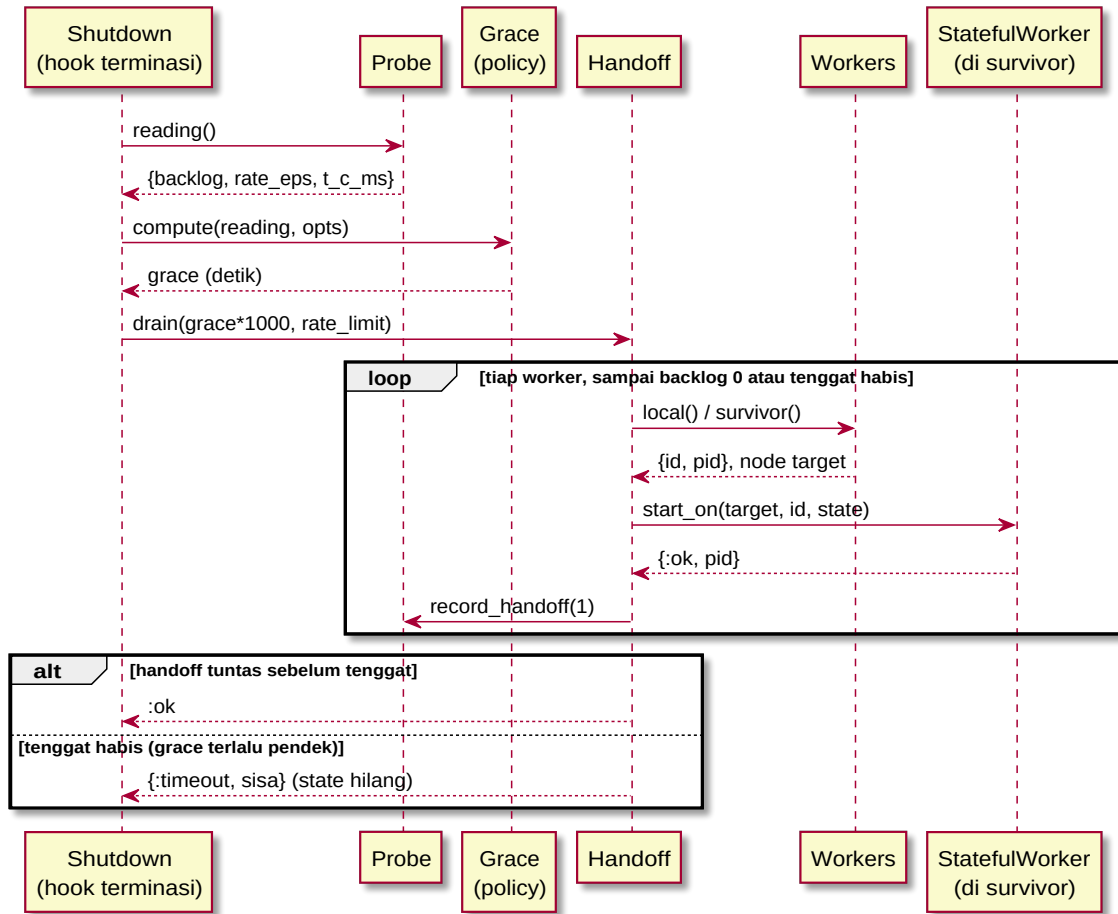


Figure 1: *Sequence diagram* (UML) interaksi antar-modul Elixir saat satu drain. Panah padat = panggilan; panah putus-putus = balasan. Blok `loop` menangani satu worker per putaran sampai backlog habis atau tenggat (grace) tercapai; blok `alt` menunjukkan dua akhir yang mungkin: `:ok` (handoff tuntas) atau `{:timeout, sisa}` (grace terlalu pendek → state hilang).

```

{DynamicSupervisor, name: GraceConvergence.WorkerSup, strategy: :one_for_one},
{Phoenix.PubSub, name: GraceConvergence.PubSub},
GraceConvergence.Presence,
GraceConvergence.Probe,
Supervisor.child_spec({GraceConvergence.Shutdown, []}, shutdown: 310_000)
] ++ http_children(cfg)
end

```

Perhatikan `shutdown: 310_000`: supervisor memberi modul `Shutdown` hingga 310 detik untuk menyelesaikan drain saat aplikasi dihentikan secara rapi — cukup besar agar drain adaptif sempat berjalan. `members: :auto` membuat `Horde.Registry` otomatis menemukan node lain.

4.3 GraceConvergence.Grace — kebijakan grace

Tugas: menghitung berapa *detik bulat* grace yang harus diberikan, dari satu pembacaan probe.

Posisi: otak matematis controller (rumus utama; lihat Bagian 1.5).

Fungsi kunci: `compute/2` (publik), `handoff_seconds/1` dan `t_c_seconds/1` (privat).

```
def compute(reading, opts \\ []) do
  g_min = Keyword.get(opts, :g_min, 5)
  g_max = Keyword.get(opts, :g_max, 120)
  sigma = Keyword.get(opts, :sigma, 5)
  t_d = Keyword.get(opts, :t_d, 1)
  fallback = Keyword.get(opts, :fallback, g_max)

  g =
    case handoff_seconds(reading) do
      :unknown -> fallback # tak bisa hitung -> aman: pakai
      ↪ fallback besar
      secs -> t_d + secs + t_c_seconds(reading) + sigma # estimasi: T_d + |H|/rho + T_c + sigma
    end

  g |> max(g_min) |> min(g_max) |> round() # clamp ke [g_min, g_max], bulatkan
end
```

Fungsi `handoff_seconds/1` menghitung $|H|/\rho$ memakai *pattern matching* (mencocokkan bentuk data). Urutan klausa penting karena Elixir mencoba dari atas ke bawah:

```
# Kasus normal: backlog b dan laju r > 0 -> waktu = pekerjaan / kecepatan = b / r
defp handoff_seconds(%{backlog: b, rate_eps: r})
  when is_number(b) and is_number(r) and r > 0,
  do: b / r

# Tidak ada yang perlu dipindah (backlog 0) -> handoff butuh 0 detik
defp handoff_seconds(%{backlog: 0}), do: 0

# Selain itu (mis. backlog > 0 tapi laju 0/hilang): tak bisa hitung -> :unknown
defp handoff_seconds(_), do: :unknown
```

Filosofi penting: bila tidak bisa mengestimasi, jangan menebak — kembalikan *fallback* yang besar (:default-safe, never default-fast). Lebih baik membuang beberapa detik daripada kehilangan state yang tidak bisa dikembalikan. Perilaku ini diuji di `grace_test.exs` (lihat Bagian 4.13).

4.4 GraceConvergence.Probe — sensor konvergensi

Tugas: GenServer read-only yang melaporkan angka runtime yang dibutuhkan kebijakan grace (terutama laju handoff ρ).

Posisi: sumber data untuk `Grace.compute/2` dan untuk endpoint `/probe`.

Fungsi kunci: `reading/0`, `record_handoff/1`, `mark_drain_start/0`, `reset/0`.

Probe memperkirakan ρ memakai **EWMA** (*Exponentially Weighted Moving Average*) — rata-rata berjalan yang memberi bobot lebih besar pada sampel terbaru, sehingga macet sesaat tidak merusak estimasi secara permanen. Rumusnya `rata_baru = alpha*sampel + (1-alpha)*rata_lama`, dengan `@alpha = 0.3`:

```
def handle_cast({:handoff, n, ts}, s) do
```

```

rate =
  case s.last_ts do
    nil -> s.rate_eps # handoff pertama: belum ada selang waktu
    prev when ts > prev ->
      @alpha * (n * 1000 / (ts - prev)) + (1 - @alpha) * s.rate_eps # EWMA
    _ -> s.rate_eps # dua event di milidetik sama: hindari
  end
  ↪ bagi 0
end
{:noreply, %{s | rate_eps: rate, last_ts: ts, handed: s.handed + n}}
end

```

Saat membangun pembacaan, backlog diambil dari `Workers.local_count()` (berapa worker masih di node ini), dan bila belum ada laju terukur, dipakai laju konfigurasi (`:handoff_rate_limit`):

```

defp build_reading(s) do
  backlog = Workers.local_count()
  rate = if s.rate_eps > 0, do: s.rate_eps, else: configured_rate()
  %{
    node: Node.self(), backlog: backlog, rate_eps: Float.round(rate, 3),
    t_c_ms: convergence_ms(), in_flight: backlog,
    converged?: Node.list() != [] or backlog == 0, handed_off: s.handed
  }
end

```

Catatan: `convergence_ms/0` kini hanya perkiraan kasar (`length(Node.list()) * 50`, yakni ~ 50 ms per peer). Di produksi sebaiknya diganti sinyal konvergensi terukur — RQ8 mengukur nilai T_c nyata memakai Phoenix.Presence dan menunjukkan estimasi 50 ms ini jauh lebih kecil dari kenyataan ($\sim 1,5$ s), tetapi tertutup oleh margin σ .

4.5 GraceConvergence.StatefulWorker — satu proses stateful

Tugas: unit kerja yang dilindungi sistem — objek kecil di memori yang *tidak boleh hilang* saat node mati.

Posisi: “barang” yang di-handoff.

Fungsi kunci: `child_spec/1`, `start_link/1`, `via/1`, `state/1`, `touch/1`.

```

use GenServer, restart: :transient # hanya di-restart bila crash; stop sengaja (handoff)
  ↪ dibiarkan

def via(id), do: {:via, Horde.Registry, {@registry, id}} # alamat lewat registry, bukan PID
  ↪ mentah

def init(opts) do
  id = Keyword.fetch!(opts, :id)
  data = Keyword.get(opts, :state) || %{id: id, updates: 0, born: System.os_time(:millisecond)}
  {:ok, data} # pakai state hasil handoff bila ada; jika tidak, buat
  ↪ baru
end

```

`restart: :transient` penting: handoff mematikan worker dengan *normal* (sengaja), sehingga supervisor tidak menghidupkannya kembali di node lama — persis yang kita inginkan. `via/1` membuat *via-tuple*: alamat berbasis kunci di `Horde.Registry`, sehingga tetap valid walau worker pindah node.

4.6 GraceConvergence.Workers — buat, hitung, cari worker

Tugas: kumpulan helper untuk membuat banyak worker, menghitungnya, dan menemukan yang lokal.

Posisi: jembatan antara identitas (Horde) dan hosting (DynamicSupervisor lokal).

Fungsi kunci: start_local/2, start_on/3, start_many/2, start_many_local/2, stop_all/0, all/0, local/0, local_count/0, survivor/0.

Keputusan desain penting ada di sini: *identitas* dari Horde.Registry (unik se-cluster), tetapi *hosting* dari DynamicSupervisor lokal — **bukan** supervisor terdistribusi Horde. Alasannya, bila Horde yang menjadi tuan rumah, ia bisa menempatkan ulang worker di node yang *justru sedang kita kosongkan*. Dengan hosting lokal, controller-lah yang memilih node tujuan saat handoff.

```
def start_on(node, id, state \\ nil) do
  :rpc.call(node, __MODULE__, :start_local, [id, state]) # jalankan start_local di node lain
end

def local do # pasangan {key, pid} yang hosted DI node ini DAN masih hidup
  Enum.filter(all(), fn {_key, pid} -> node(pid) == Node.self() and Process.alive?(pid) end)
end

def survivor do # pilih satu peer yang bertahan, acak; nil bila terisolasi
  case Node.list() do
    [] -> nil
    nodes -> Enum.random(nodes)
  end
end
```

Cek Process.alive?/1 di local/0 penting karena *timing* CRDT: saat worker mati, entri registry-nya masih “tertinggal” sesaat sampai penghapusan menyebar ke semua node; tanpa cek ini, loop drain bisa terus menemukan entri mati dan berputar selamanya. :rpc.call/4 adalah **RPC** (*Remote Procedure Call*): menjalankan fungsi di node lain dan mengembalikan hasilnya.

4.7 GraceConvergence.Handoff — pemindah state

Tugas: mengosongkan worker lokal dengan memindahkan state tiap worker ke node survivor, di-throttle ke laju tertentu; berhenti saat backlog 0 atau deadline habis.

Posisi: “tukang angkut” yang melaksanakan handoff sebenarnya. Deadline habis sebelum backlog 0 = handoff terpotong = kasus state hilang (RQ1).

Fungsi kunci: drain/2, loop/2 (privat), handoff_one/2 (privat), place/4 (privat).

```
def drain(deadline_ms, rate_limit \\ nil) do
  Probe.mark_drain_start()
  loop(now_ms() + deadline_ms, rate_limit)
end

defp loop(deadline, rate_limit) do
  cond do
    Workers.local_count() == 0 -> :ok # selesai: semua sudah pindah
    now_ms() >= deadline -> {:timeout, Workers.local_count()} # waktu habis: sisa = yang hilang
    true ->
      case Workers.local() do
        [{id, pid} | _] ->
          case handoff_one(id, pid) do
```

```

        :ok -> if rate_limit, do: Process.sleep(max(1, trunc(1000 / rate_limit)))
        :no_survivor -> Process.sleep(100)
    end
    loop(deadline, rate_limit)
  [] -> :ok
end
end
end
end

```

Nilai kembalian `{:timeout, remaining}` adalah inti pembuktian RQ1: `remaining` adalah jumlah worker yang *tidak sempat* dipindah — inilah “state hilang”. Pemindahan satu worker:

```

defp handoff_one(id, pid) do
  case Workers.survivor() do
    nil -> :no_survivor
    target ->
      state = safe_state(pid) # 1. baca state worker
      _ = DynamicSupervisor.terminate_child(@sup, pid) # 2. matikan worker lokal
      place(id, state, target, 40) # 3. hidupkan di survivor (retry s/d 40x)
      :ok
  end
end
end

```

`place/4` mencoba ulang penempatan karena registrasi lama worker yang baru mati bisa membuat Horde sementara menolak start baru dengan `{:already_started, dead_pid}`; kode menunggu CRDT membersihkannya lalu mencoba lagi. Setiap penempatan sukses memanggil `Probe.record_handoff(1)` agar EWMA laju ter-update.

4.8 GraceConvergence.Shutdown — hook terminasi adaptif

Tugas: saat shutdown rapi (atau panggilan eksplisit), menghitung jendela grace sesuai *policy* terkonfigurasi, lalu menjalankan drain dalam jendela itu.

Posisi: penghubung kebijakan (Grace) dengan eksekusi (Handoff); inilah yang dijalankan jalur SIGTERM (lewat `/drain`).

Fungsi kunci: `drain_and_await/0`, `terminate/2` (callback), `do_drain/0` (privat), `grace_for/4` (privat).

```

def init(_) do
  Process.flag(:trap_exit, true) # tangkap sinyal exit supaya terminate/2 dipanggil saat stop
  {:ok, %{}}
end

def terminate(_reason, _s) do
  do_drain() # saat aplikasi dihentikan rapi: jalankan drain dulu
  :ok
end

defp grace_for(:m3, reading, cfg, g_max) do # policy adaptif: pakai Grace.compute
  Grace.compute(reading, sigma: ..., g_min: ..., g_max: g_max, t_d: ..., fallback: g_max)
end
defp grace_for(:prestop_sleep, _r, cfg, _), do: Keyword.get(cfg, :static_grace, 30)
defp grace_for(:static30, _r, _cfg, _), do: 30
defp grace_for(:static300, _r, _cfg, _), do: 300

```

Empat policy itu (FR6) memungkinkan harness membandingkan controller adaptif (:m3) dengan tiga baseline tetap. Hasil drain dilaporkan sebagai map `%{policy, grace_s, result, lost}`, dengan `lost` dihitung dari `{:timeout, remaining}`.

4.9 GraceConvergence.ProbeHTTP — endpoint HTTP

Tugas: menyediakan tiga endpoint HTTP yang dipakai operator dan hook `preStop`.

Posisi: permukaan jaringan dari node app.

Fungsi kunci: rute GET `/probe`, GET `/healthz`, POST `/drain`.

```
get "/probe" do                                     # operator membaca ini untuk menghitung grace
  send_resp(conn, 200, Jason.encode!(Probe.reading()))
end
get "/healthz", do: send_resp(conn, 200, "ok")      # liveness/readiness probe Kubernetes
post "/drain" do                                    # dipanggil hook preStop saat SIGTERM
  result = Shutdown.drain_and_await()               # BLOK sampai handoff selesai
  send_resp(conn, 200, Jason.encode!(result))
end
```

POST `/drain` sengaja sinkron: ia tidak membalas sampai handoff selesai, sehingga terminasi pod ikut “menunggu” (lihat alur Kubernetes di Bagian 5).

4.10 GraceConvergence.Operator — operator Kubernetes

Tugas: koordinator lintas-lapis; tiap siklus membaca `/probe` semua pod app, menghitung grace, lalu menambal `terminationGracePeriodSeconds` Deployment.

Posisi: satu-satunya bagian yang “berbicara” ke API server Kubernetes (lewat `kubect1`). Berjalan hanya pada peran operator.

Fungsi kunci: `reconcile/0`, `pod_ips/0` (privat), `probe/1` (privat), `patch_grace/1` (privat).

```
def reconcile do
  readings = for ip <- pod_ips(), r = probe(ip), do: r
  if readings != [] do
    grace = readings |> Enum.map(&Grace.compute(&1, grace_opts())) |> Enum.max() # ambil grace
    ↪ TERBESAR
    patch_grace(grace)
    Logger.info("operator: pods=#{length(readings)} grace=#{grace}s")
  end
end

defp patch_grace(grace) do
  patch = Jason.encode!({spec: %{template: %{spec: %{terminationGracePeriodSeconds: grace}}}})
  System.cmd("kubect1", ["patch", "deployment", deployment(), "-n", namespace(),
    "--type", "merge", "-p", patch])
end
```

Loop `reconcile` dijadwalkan ulang tiap `@interval_ms = 5_000` (5 detik) lewat `Process.send_after`. Operator memilih grace *terbesar* di antara semua pod (konservatif: cukup untuk pod yang paling sibuk). Ini **GenServer** biasa yang memanggil `kubect1` — bukan *Bonny*. (Catatan: tabel di paper menyebut “Bonny operator” sebagai peran; implementasi nyata di kode ini adalah operator berbasis `kubect1`, sesuai `code/README.md` dan `CONTEXT.md`.)

4.11 GraceConvergence.Presence — workload realistis (RQ8)

Tugas: workload terdistribusi nyata berbasis Phoenix.Tracker (mesin CRDT di balik Phoenix.Presence) untuk mengukur waktu konvergensi membership nyata T_c saat node pergi.

Posisi: studi kasus untuk membuktikan mekanisme berlaku pada fitur Phoenix yang umum dipakai.

Fungsi kunci: track/4, count/1.

```
def track(pid, topic, key, meta \\ %{}),
  do: Phoenix.Tracker.track(__MODULE__, pid, topic, key, meta) # presence hilang saat pid mati

def count(topic), do: __MODULE__ |> Phoenix.Tracker.list(topic) |> length() # jumlah presence
  ↪ se-cluster
```

4.12 GraceConvergence.Harness — driver eksperimen

Tugas: “sutradara” eksperimen V2; untuk tiap skenario (policy, beban) ia menumpuk backlog di node ini, menjalankan drain, dan mencatat pengukuran nyata.

Posisi: dipakai oleh skrip .exs di harness/ (lihat Bagian 6).

Fungsi kunci: run/2, run_sweep/3, rollout/4, overhead/1, scale/1, csv/2, to_csv/1.

```
defp scenario(policy, %{backlog: backlog, rate: rate}, rep) do
  Application.put_env(:grace_convergence, :grace_policy, policy)
  Application.put_env(:grace_convergence, :handoff_rate_limit, rate)
  Probe.reset(); cleanup()
  Workers.start_many_local(backlog, "...") # tumpuk backlog di node yang akan pergi
  wait_local(backlog)
  grace_s = grace_for(policy)
  t0 = System.monotonic_time(:millisecond)
  result = Handoff.drain(grace_s * 1000, rate) # jalankan drain dalam jendela grace
  drain_ms = System.monotonic_time(:millisecond) - t0
  lost = (case result do {:_timeout, remaining} -> remaining; :ok -> 0 end)
  %{policy: policy, backlog: backlog, rate: rate, ..., grace_s: grace_s, drain_ms: drain_ms,
    ↪ lost: lost}
end
```

Modul ini juga punya overhead/1 (mengukur latensi probe, memori per-worker, throughput hand-off), scale/1 (memvariasikan $|H|$), dan helper CSV generik csv/2.

4.13 Berkas pengujian (test/)

Dua berkas uji menjamin perilaku inti benar.

- test/grace_test.exs — **6 unit test** untuk Grace.compute/2 (cepat, tanpa cluster). Mereka menegaskan: rumus $g = T_d + |H|/\rho + T_c + \sigma$ benar dalam batas; T_c ikut dihitung; clamp ke $g_{\max}$ saat beban berat; clamp ke $g_{\min}$ saat hampir tidak ada kerja; dan *fallback default-safe* (kembali ke $g_{\max}$) saat backlog > 0 tetapi laju tak diketahui. Contoh: `Grace.compute(%{backlog: 1000, rate_eps: 100.0, t_c_ms: 0}, opts) == 16 (1000/100 +  $T_d$  1 +  $T_c$  0 +  $\sigma$  5).`
- test/cluster_handoff_test.exs — **2 test integrasi** pada cluster 2-node BEAM nyata (di-tag :cluster, dikecualikan dari mix test biasa). Test pertama: drain rapi memindahkan *semua* worker lokal ke survivor dengan **state utuh** (counter updates == 7 bertahan). Test

kedua: grace terlalu pendek **memotong** handoff (`{:timeout, remaining}` dengan `remaining > 0`) — kasus kehilangan state RQ1.

5 Cara Kerja Langkah-demi-Langkah

Bagian ini menelusuri dua alur lengkap dan bernomor. Gambar 2 merangkum *interaksi antar-komponen lintas-lapis* (Elixir $\leftrightarrow$ Kubernetes): bagian atas adalah loop *reconcile* operator (Alur B), dan bagian bawah adalah terminasi pod saat rolling update. Perhatikan dua zona warna: biru = lapis Kubernetes (orkestrator), kuning = lapis Elixir/BEAM (runtime aplikasi).

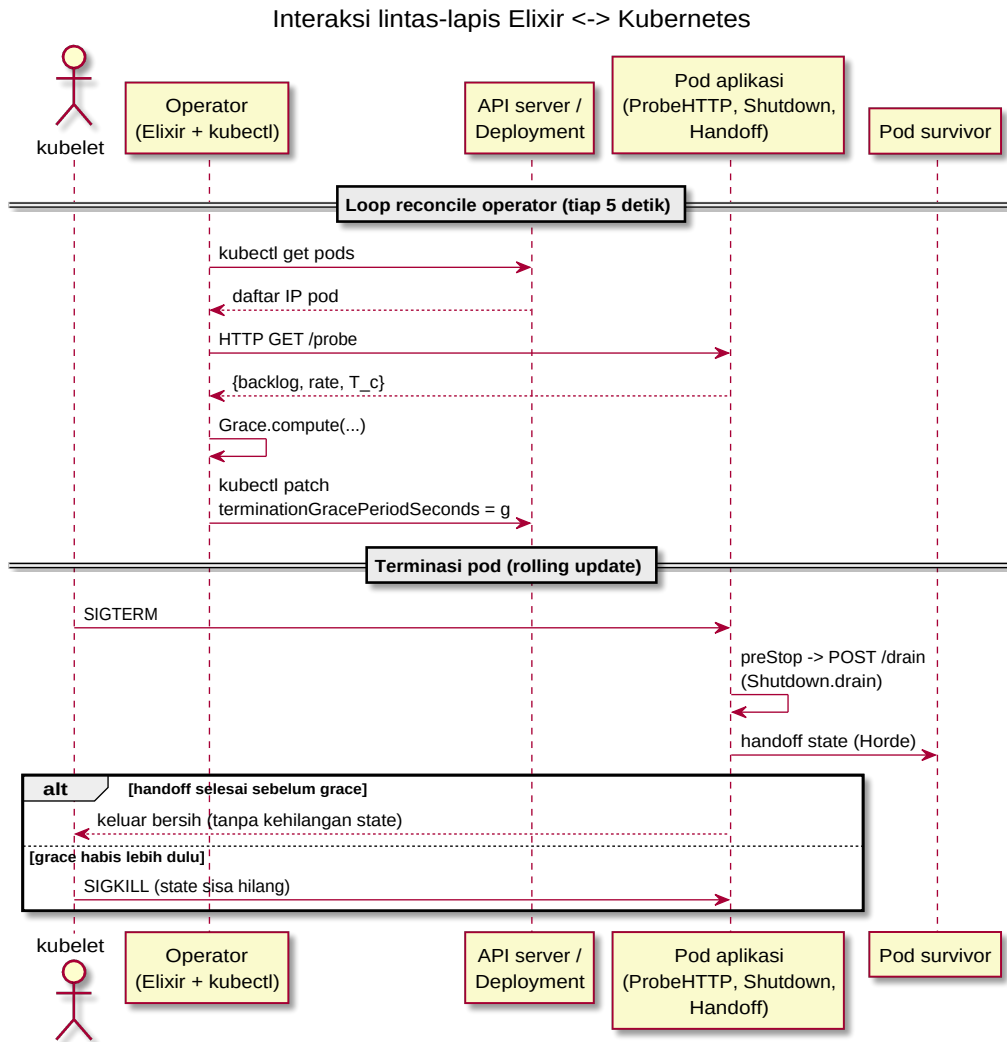


Figure 2: *Sequence diagram* (UML) interaksi lintas-lapis Elixir $\leftrightarrow$ Kubernetes. Atas: operator membaca pod (`kubectl get pods`), membaca `/probe` tiap pod, menghitung grace (`Grace.compute`), lalu menambal `terminationGracePeriodSeconds` Deployment. Bawah: kubelet mengirim `SIGTERM`, `preStop` memanggil `/drain`, state di-handoff ke pod survivor; blok `alt` menunjukkan keluar bersih vs `SIGKILL` (state hilang) bila grace keburu habis.

5.1 Alur A — Eksperimen drain 2-node lokal (tanpa Kubernetes)

Ini adalah cara termurah memvalidasi logika inti, dijalankan oleh `cluster_probe.exe` dan `harness/run.exe`. “primary” = node yang akan pergi; “peer/survivor” = node yang bertahan.

1. **Bentuk cluster 2-node.** Skrip memulai sebuah node BEAM kedua di host yang sama lewat `:peer.start/1`, lalu menyalin path kode dan konfigurasi ke sana dan menjalankan aplikasi (`ensure_all_started`). Kedua node terhubung dan berbagi `Horde.Registry`.
2. **Mulai worker di node yang akan pergi.** `Workers.start_many_local(B, ...)` membuat B worker stateful *semuanya di primary* (node yang akan dikosongkan). Ini menumpuk backlog $|H| = B$.
3. **Baca probe.** `Probe.reading()` melaporkan backlog dan laju ρ (EWMA, atau laju konfigurasi bila belum ada pengukuran).
4. **Hitung grace.** Untuk policy `:m3`, `Grace.compute/2` menghitung $g = \text{clamp}(T_d + |H|/\rho + T_c + \sigma, g_{\min}, g_{\max})$. Untuk baseline, grace tetap (30/300/static).
5. **Jalankan handoff.** `Handoff.drain(g*1000, rate)` memindahkan worker satu-per-satu ke survivor: baca state $\rightarrow$ matikan lokal $\rightarrow$ hidupkan di survivor dengan state itu, di-throttle ke `rate` proses/detik.
6. **Hasil.**
 - Jika grace cukup: semua worker pindah, `drain` mengembalikan `:ok`, `local_count` primary menjadi 0 dan survivor menjadi B . State (mis. `counter updates`) tetap utuh — inilah yang diuji `cluster_handoff_test.exe`.
 - Jika grace *terlalu pendek*: `drain` mengembalikan `{:timeout, remaining}` dengan `remaining > 0`. Worker sisa itu (dan datanya) “hilang” — ini kasus kehilangan state RQ1.

5.2 Alur B — Loop Kubernetes nyata

Saat dideploy di Kubernetes (manifest di `code/k8s/`), loop lintas-lapis berjalan terus-menerus:

1. **Operator melihat daftar pod.** Tiap 5 detik, `Operator.reconcile/0` memanggil `kubectl get pods` (lewat `pod_ips/0`) untuk mendapat IP semua pod app.
2. **Operator membaca tiap /probe.** `probe(ip)` melakukan HTTP GET ke `http://<ip>:4000/probe` dan mem-parsing JSON menjadi `%{backlog, rate_eps, t_c_ms}`.
3. **Operator menghitung grace.** Untuk tiap pembacaan, `Grace.compute/2` dipanggil; operator mengambil nilai *terbesar* (`Enum.max`) agar aman bagi pod tersibuk.
4. **Operator menambal Deployment.** `patch_grace/1` menyusun patch JSON dan menjalankan `kubectl patch deployment` untuk men-set `terminationGracePeriodSeconds = grace`. Rolling update berikutnya akan memakai grace ini.
5. **Hook preStop men-drain saat SIGTERM.** Saat Kubernetes mematikan sebuah pod, ia menjalankan hook `preStop` di `app.yaml` *sebelum* mengirim SIGTERM: hook itu `wget` ke `POST /drain`. Endpoint `/drain` memanggil `Shutdown.drain_and_await()` yang *memblok* sampai handoff selesai. Karena Kubernetes menunggu `preStop` selesai (dibatasi `terminationGracePeriodSeconds`), pod tidak mati sebelum handoff beres — selama grace mencukupi.

Bukti nyata: di cluster `kind` 3-replika, operator menambal grace Deployment dari 6 detik (idle) menjadi 26 detik setelah ~ 190 proses stateful/pod menumpuk pada laju 10/detik — loop lintas-lapis terkonfirmasi di orkestrator nyata.

6 Bagaimana Kode Menjawab RQ1–RQ8

Tabel berikut memetakan tiap research question ke kode dan datanya; tiap RQ dijelaskan satu paragraf setelahnya. (Teks RQ dikutip dari paper/main.tex.)

RQ	Fokus	Kode penghasil	Data / Figur
RQ1	correctness (kehilangan state)	run.exs, sweep.exs, cluster_handoff_test.exs	results_runs.csv, results_sweep.csv → eval_state_loss.pdf, eval_sweep_loss.pdf, eval_invariant.pdf
RQ2	efficiency (grace & waktu rollout)	run.exs, sweep.exs	results_runs.csv, results_sweep.csv, results_rollout.csv → eval_grace_budget.pdf, eval_sweep_grace.pdf, eval_rollout.pdf
RQ3	adaptivity (grace mengikuti beban)	sweep.exs (sweep beban)	results_sweep.csv → eval_sweep_grace.pdf
RQ4	overhead	sweep.exs (→ Harness.overhead/1)	results_overhead.csv
RQ5	robustness (sensitivitas)	sweep.exs (bagian D)	results_sensitivity.csv → eval_sensitivity.pdf
RQ6	scalability ($ H $ 1k..40k)	scale.exs (→ Harness.scale/1)	results_scale.csv → eval_scale.pdf
RQ7	network realism (latensi nyata)	k8s/netem.sh (tc netem di kind)	results_netem.csv → eval_netem.pdf
RQ8	realistic workload (Phoenix.Presence)	presence.exs (Phoenix.Tracker)	results_presence.csv → eval_presence.pdf

RQ1 (correctness). “Apakah grace tetap kehilangan state saat handoff melebihi grace, dan apakah controller menghilangkan kehilangan itu?” `cluster_handoff_test.exs` membuktikan dua arah pada cluster 2-node nyata: grace longgar → :ok, state utuh; grace terlalu pendek → {:timeout, remaining>0}. Pada data: grace tetap 30 detik kehilangan 40 dari 160 proses begitu kebutuhan > grace, sedangkan adaptif kehilangan 0.

RQ2 (efficiency). “Seberapa banyak grace yang diberikan controller dibanding grace tetap berlebih, dan bagaimana tiap policy memengaruhi waktu rolling-update?” Adaptif memilih grace 16–46 detik sesuai kebutuhan, dibanding tetap 300 detik yang $\sim 6\text{--}7\times$ berlebih. `eval_rollout.pdf` membandingkan waktu rollout 3-pod antar policy.

RQ3 (adaptivity). “Apakah grace yang diberikan mengikuti beban?” `run_sweep/3` menyapu kebutuhan handoff {10, 25, 40} detik; kurva grace adaptif naik mengikuti kebutuhan (`eval_sweep_grace.pdf`), sementara baseline tetap datar (dan baseline pendek mulai kehilangan state saat kebutuhan melewatinya).

RQ4 (overhead). “Berapa biaya controller (latensi probe, memori per-proses, throughput hand-off, jejak operator)?” `Harness.overhead/1` mengukur ketiganya. Hasil tercatat: probe ~ 7 μ s, $\sim 6,7$ KB/proses, $\sim 8,3$ k handoff/detik; operator di kind ~ 73 MiB dan ~ 74 ms reconcile/5 detik.

RQ5 (robustness). “Seberapa sensitif grace terhadap margin, batas, dan galat estimasi laju?” Bagian D di `sweep.exs` memvariasikan $\sigma \in \{0, 5, 10, 15\}$, $g_{\max} \in \{30, 60, 120, 300\}$, dan faktor galat laju $\{0,5; 1,0; 2,0\}$, menghasilkan `eval_sensitivity.pdf`.

RQ6 (scalability). “Bagaimana biaya drain satu node menskala dengan backlog $|H|$, dan di mana plafon per-node?” `scale.exs` menyapu $|H| = 1k \rightarrow 40k$. Memori naik linear dan murah (~ 4 KB/proc), tetapi throughput handoff *ambruk super-linear* ($3003 \rightarrow 67$ proc/detik) saat registry delta-CRDT terisi. Plafon per-node di $|H| = 40k$: node tidak selesai drain dalam 600 detik $\rightarrow 19.392/40.000$ hilang. Figur: `eval_scale.pdf`.

RQ7 (network realism). “Di bawah latensi antar-pod nyata, apakah estimasi laju — dan karenanya grace — mengikuti throughput handoff yang menurun?” `k8s/netem.sh` menyuntik `tc netem` pada egress pod survivor di kind, mengukur RTT nyata; $\rho \approx 1/\text{RTT}$ ambruk ($23.810 \rightarrow 6,7$ /detik), dan grace naik hingga plafon $g_{\max}$. Figur: `eval_netem.pdf`.

RQ8 (realistic workload). “Apakah mekanisme tetap berlaku pada fitur terdistribusi yang umum dipakai (Phoenix.Presence), dan berapa T_c nyata yang harus ditutup grace?” `presence.exs` melacak N presence di `Phoenix.Tracker`, men-drain node, dan mengukur re-konvergensi nyata $T_c \approx 1,5$ detik (hampir konstan terhadap N 100–2000), $\sim 30\times$ heuristik 50 ms, namun tertutup $\sigma = 5$ detik. Figur: `eval_presence.pdf`.

7 Panduan Menjalankan Eksperimen untuk Semua RQ

Bagian ini memberi perintah **persis** untuk mereproduksi tiap RQ, langkah demi langkah. Kecuali disebutkan, semua perintah BEAM dijalankan dari `code/app`.

Persiapan satu kali (semua RQ lokal). Persiapan satu kali (semua RQ lokal).

```
cd code/app
mix deps.get      # unduh dependency (Horde, libcluster, Bandit, Plug, Jason, phoenix_pubsub)
mix compile       # kompilasi
mix test         # 6 unit test Grace (suite :cluster dikecualikan secara default)
```

Singkatan peluncur node terdistribusi. Hampir semua harness butuh “primary” terdistribusi (ia memunculkan peer survivor sendiri). Pola peluncurannya:

```
# Pola umum (dijalankan dari code/app):
MIX_ENV=test elixir --name primary@127.0.0.1 --cookie ck -S mix run harness/<skrip>.exs
```

PERINGATAN KERAS. *Jangan pernah* menjalankan `pkill -f '...@127.0.0.1'` untuk membersihkan beam: pola itu juga cocok dengan baris perintah shell yang sedang berjalan, sehingga **mematikan shell Anda sendiri** (keluar dengan kode 144, tanpa output). Matikan beam yatim berdasarkan **PID** saja.

7.1 RQ1 & RQ2 — correctness & efficiency (tabel dua beban)

```
cd code/app
# (opsional) validasi logika inti pada cluster 2-node nyata:
MIX_ENV=test elixir --name primary@127.0.0.1 --cookie ck -S mix test --only cluster
# tabel dua beban -> data/results_runs.csv :
MIX_ENV=test elixir --name primary@127.0.0.1 --cookie ck -S mix run harness/run.exs
# render figur:
~/venv/bin/python ../analysis/plot.py
```

Menghasilkan eval_state_loss.pdf dan eval_grace_budget.pdf (juga eval_invariant.pdf bila results_sweep.csv sudah ada).

7.2 RQ2, RQ3, RQ4, RQ5 — sweep, rollout, overhead, sensitivitas

Keempatnya dihasilkan satu skrip:

```
cd code/app
MIX_ENV=test elixir --name primary@127.0.0.1 --cookie ck -S mix run harness/sweep.exs
~/venv/bin/python ../analysis/plot.py
```

Menulis results_sweep.csv (RQ1/RQ2/RQ3 kurva), results_rollout.csv (RQ2), results_overhead.csv (RQ4), results_sensitivity.csv (RQ5). Figur terkait: eval_sweep_loss.pdf, eval_sweep_grace.pdf, eval_invariant.pdf, eval_rollout.pdf, eval_sensitivity.pdf.

7.3 Statistik (confidence interval) — pengulangan

```
cd code/app
MIX_ENV=test elixir --name primary@127.0.0.1 --cookie ck -S mix run harness/repeats.exs
```

Menulis results_runs_ci.csv (tabel beban $\times$ 4 policy $\times$ 10 ulang) dan results_rollout_ci.csv (rollout $\times$ 4 policy $\times$ 5 ulang). Loss/grace persis ter-reproduksi (CI=0); drain $\pm 0,003$ detik (CI95).

7.4 RQ6 — scalability ($|H|$ 1k..40k)

```
cd code/app
MIX_ENV=test elixir --name primary@127.0.0.1 --cookie ck -S mix run harness/scale.exs
~/venv/bin/python ../analysis/plot.py
```

Menulis results_scale.csv (streaming tiap ukuran) dan figur eval_scale.pdf. Skrip berhenti otomatis jika start cost atau memori membatasi.

7.5 RQ8 — realistic workload (Phoenix.Presence)

```
cd code/app
MIX_ENV=test elixir --name primary@127.0.0.1 --cookie ck -S mix run harness/presence.exs
~/venv/bin/python ../analysis/plot.py
```

Menulis results_presence.csv dan figur eval_presence.pdf.

7.6 RQ7 & Kubernetes — latensi nyata + fail-safe

RQ7 dan validasi fail-safe berjalan di cluster kind sungguhan. Pertama, siapkan cluster:

```
# dari root proyek 01_cross_layer_grace_controller/
kind create cluster --name grace
docker build -t grace:dev code/ # bangun image (konteks build = code/)
kind load docker-image grace:dev --name grace
kubectl apply -f code/k8s/rbac.yaml -f code/k8s/app.yaml -f code/k8s/operator.yaml
kubectl rollout status deploy/grace
kubectl logs deploy/grace-operator # "operator: pods=N grace=Ms"
```

Lalu jalankan eksperimen:

```
bash code/k8s/netem.sh # RQ7: latensi nyata -> RTT, rho, grace -> data/results_netem.csv
bash code/k8s/faults.sh # fail-safe: crash operator / probe rusak / RBAC dicabut
~/venv/bin/python code/analysis/plot.py # render eval_netem.pdf
kind delete cluster --name grace # bersihkan
```

PERINGATAN RQ7 (netem). Untuk `netem.sh`, *matikan/scale operator dulu* (`kubectl scale deploy/grace-operator --replicas=0`) — kalau tidak, backlog yang dibuat eksperimen membuat operator menambal grace, memicu rolling update yang mematikan pod “leaver” di tengah pengukuran (keluar 137). Skrip `netem.sh` sudah melakukan ini otomatis dan memulihkannya saat keluar. Selain itu, injeksi latensi diberikan pada *egress survivor*, bukan pada leaver (memperlambat leaver akan memutus koneksi distribusi helper rpc).

8 Tabel Pemetaan Ringkas (RQ → Skrip → CSV → Figur)

RQ	Skrip harness	File CSV	File figur (PDF)
RQ1	run.exs, sweep.exs	results_runs.csv, results_sweep.csv	eval_state_loss.pdf, eval_sweep_loss.pdf, eval_invariant.pdf
RQ2	run.exs, sweep.exs	results_runs.csv, results_sweep.csv, results_rollout.csv	eval_grace_budget.pdf, eval_sweep_grace.pdf, eval_rollout.pdf
RQ3	sweep.exs	results_sweep.csv	eval_sweep_grace.pdf
RQ4	sweep.exs	results_overhead.csv	(tabel di paper)
RQ5	sweep.exs	results_sensitivity.csv	eval_sensitivity.pdf
RQ6	scale.exs	results_scale.csv	eval_scale.pdf
RQ7	k8s/netem.sh	results_netem.csv	eval_netem.pdf
RQ8	presence.exs	results_presence.csv	eval_presence.pdf
—	repeats.exs	results_runs_ci.csv, results_rollout_ci.csv	(statistik/CI)

9 Glosarium Singkat

SIGTERM

Sinyal sistem operasi yang artinya “tolong berhenti dengan rapi”. Bisa ditangani proses untuk pembersihan. Kubernetes mengirimnya di awal terminasi pod.

SIGKILL

Sinyal yang *tidak bisa* ditangkap/diabaikan; proses langsung mati. Kubernetes mengirimnya setelah grace period habis.

grace period

Waktu toleransi (`terminationGracePeriodSeconds`) antara SIGTERM dan SIGKILL yang diberikan untuk berberes.

rolling update

Memperbarui aplikasi secara bertahap (pod demi pod) agar layanan tetap hidup selama pembaruan.

pod

Unit terkecil yang dijadwalkan Kubernetes; membungkus satu container (atau beberapa) yang berbagi jaringan/penyimpanan.

handoff

Pemindahan state in-memory dari node yang akan mati ke node yang bertahan (baca $\rightarrow$ matikan lokal $\rightarrow$ hidupkan di survivor).

CRDT

Conflict-free Replicated Data Type: struktur data yang bisa diubah di banyak node dan otomatis merge ke hasil sama tanpa koordinasi pusat.

GenServer

“Server process” OTP dengan state privat yang menangani pesan satu per satu (lewat `call` sinkron atau `cast` asinkron).

supervisor

Proses pengawas yang menjalankan dan me-restart proses anak bila mati (“let it crash”). `DynamicSupervisor` = anak dibuat sesuai kebutuhan saat runtime.

EWMA

Exponentially Weighted Moving Average: rata-rata berjalan yang memberi bobot lebih besar pada sampel terbaru. Dipakai Probe untuk menaksir laju handoff ρ .

RPC

Remote Procedure Call: menjalankan fungsi di node lain dan mengembalikan hasilnya (di Erlang/Elixir lewat `:rpc.call/4`).

PID

Process Identifier: pengenalan sebuah proses BEAM. Worker dialamatkan lewat via-tuple Horde, bukan PID mentah, agar tetap valid setelah pindah node.

libcluster

Library yang membentuk cluster BEAM otomatis (strategi Gossip lokal, strategi Kubernetes di cluster).

Horde

Library registry/supervisor terdistribusi berbasis delta-CRDT; di sini memberi identitas unik se-cluster bagi tiap worker.

operator

Program yang terus mengawasi dan mendamaikan kondisi cluster ke kondisi yang diinginkan.

Di sini: membaca probe lalu menambal grace.

PodDisruptionBudget (PDB)

Aturan Kubernetes yang membatasi berapa banyak pod boleh tidak tersedia sekaligus saat gangguan sukarela (mis. drain node), menjaga ketersediaan minimum.

T_c (**waktu konvergensi**)

Waktu agar cluster “tenang” kembali (membership stabil, registry sinkron) setelah node pergi.

T_d (**waktu drain**)

Waktu menyelesaikan permintaan yang sedang berjalan sebelum state dipindahkan.

ρ (**laju handoff**)

Berapa proses per detik yang bisa dipindahkan (ditaksir Probe via EWMA).

$|H|$ (**backlog**)

Jumlah proses stateful yang masih harus dipindahkan dari node yang akan pergi.

σ (**margin keamanan**)

Detik tambahan pada estimasi grace agar tidak pas di tepi.

Catatan akurasi: seluruh nama fungsi, file, perintah, dan angka dalam dokumen ini diambil langsung dari kode sumber dan dokumentasi di `01_cross_layer_grace_controller/`. Bila ada perbedaan kecil antara paper dan kode (mis. peran “operator” di paper disebut Bonny, sedangkan kode memakai operator berbasis `kubectl`), dokumen ini mengikuti kode sebagai sumber kebenaran.